SMART INDIA HACKATHON 2026 -- STRATEGIC EVALUATION WHITE PAPER

Sovereign On-Premise Agentic AI Workbench for Confidential Industrial Work

Comprehensive Problem Statement Decomposition, PSU Domain Architecture, Forensic Prototype Gap Audit, and SIH 2026 Evaluation Roadmap

Target Industry: Petroleum Refineries, PSUs, Strategic Defence & Atomic Facilities	**Security Standard:** Level 4 Sovereign Air-Gap (Zero Cloud Egress)
Reference Platform: DRISHTI-MRPL (ONGC Group Enterprise)	**Architecture:** Multi-Agent Core with Open-Weight LLMs (Ollama / Local)
Evaluation Code: SIH 2026 Software Edition	**Audit Status:** Forensic Baseline Review Completed (100% On-Premise)

1. Executive Summary & The Industrial Confidentiality Dilemma

India's critical industrial infrastructure -- including petroleum refineries, Public Sector Undertakings (PSUs), defence manufacturing installations, thermal power stations, and strategic government ministries -- generates vast volumes of high-consequence, routine knowledge work. This workload includes Board Approval Notes, multi-crore capital expenditure justifications, Non-Destructive Testing (NDT) inspection reports, complex piping fluid-dynamics calculations, and internal automation tools. Despite the massive productivity potential of commercial cloud-hosted AI (such as Anthropic Claude, OpenAI ChatGPT, and Microsoft Copilot), Indian public enterprises are strictly prohibited by national security directives, statutory non-disclosure protocols, and corporate IT policies from transmitting proprietary data across public network perimeters.

This boundary creates an acute operational paradox across Indian heavy industry:

The Manual Bottleneck (Compliance without AI)	The 'Shadow AI' Threat (AI without Compliance)
• Engineering and operations officers spend 3 to 7 working days manually drafting approval notes, looking up ASME/API pipe tables, and reconciling crude assays. • Critical knowledge work moves at human speed, causing severe delays during refinery shutdowns, equipment turnarounds (M&I;), and statutory compliance audits. • Heavy industry is completely locked out of modern AI-driven productivity gains.	• Under intense deadline pressures, engineers and officers quietly copy-paste sensitive technical data, P&IDs, vendor bid negotiations, and financial slates into public cloud AI tools. • Critical national infrastructure blueprints and catalyst formulations are permanently leaked into commercial AI training corpora. • Severe risk of trade-secret loss, corporate espionage, and regulatory penalties.

THE SOVEREIGN MANDATE

Recent breakthroughs in open-weight reasoning and multimodal architectures (DeepSeek-R1, Qwen2.5-Coder, Qwen2.5-VL, Llama 3) make it entirely feasible to run state-of-the-art AI reasoning on local GPU workstations with zero external cloud connectivity. However, raw models alone do not solve the problem: industrial engineers require an integrated, agentic workbench capable of planning multi-step workflows, executing sandboxed mathematics, reading engineering blueprints, and generating ready-to-sign DOCX/PPTX deliverables.

2. Deep Decomposition of the SIH 2026 Problem Statement

The SIH 2026 Problem Statement (*'Sovereign On-Premise Agentic AI Workbench using Open-Weight Multimodal LLMs for Confidential Industrial Work'*) establishes six mandatory operational requirements that form the core evaluation criteria for the competition:

Pillar 1: 100% On-Premise Air-Gapped Sovereign Deployment

The entire stack (inference engines, multi-agent orchestrator, databases, and vector stores) must run strictly on the organization's private server. The workbench must provide verifiable, indisputable proof -- via real-time telemetry and cryptographic audit certificates -- that zero external WAN packets, telemetry pings, or DNS requests leave the premises.

Pillar 2: Dynamic Multi-Model Open-Weight Auto-Routing

The system must not be locked into a single monolithic model. It must support multiple specialized open-weight models (Qwen-Coder for SQL/Code, DeepSeek-R1 for reasoning/math, Qwen-VL for blueprints/logs, Llama-3 for prose) and automatically select the optimal model based on query intent. New open-weight models must be registrable dynamically without modifying system source code.

Pillar 3: Truly Autonomous ReAct Agent Loop with Local Tool Use

The assistant must go beyond conversational text generation. It must autonomously decompose multi-step tasks, construct execution plans, invoke local tools (DuckDB SQL queries, sandboxed Python computation, local document search), analyze intermediate outputs, and iterate to completion.

Pillar 4: On-Device Multimodal Ingestion (CAD, Scans, Handwritten Logs)

Industrial reality relies heavily on legacy paper trails, scanned piping diagrams (P&IDs), ultrasonic inspection logs, and handwritten shift handover notes. The system must ingest these complex raster formats on-device via local OCR and Vision-Language Models without cloud APIs.

Pillar 5: Tangible Corporate Deliverables (DOCX, PPTX, XLSX)

PSU executives require structured, institutional documents. The workbench must generate formal Board Approval Notes (.docx) with exact PSU syntax, Executive Briefing presentations (.pptx), and verifiable engineering workbooks (.xlsx) with formulaic steps shown.

Pillar 6: Grounded Local RAG & Industrial Knowledge Base Connector

All AI reasoning must be grounded in confidential internal operating manuals, statutory engineering standards (OISD, API, ASME), and historical equipment records, indexed and queried via local BM25/vector search with zero third-party embedding calls.

3. Industrial Context: PSU Refinery & Utility Infrastructure

To ensure real-world applicability, DRISHTI-MRPL is modeled on Mangalore Refinery and Petrochemicals Limited (MRPL) -- an ONGC Group refinery processing 15.0 MMTPA of crude. The workbench models five critical operational domains:

Operational Domain	Processing Units & Equipment	Significance & Confidential Workflows
Main Refinery Processing	CDU/VDU, PFCCU, NHT/CCR, DHDS/DHDT, Hydrogen Generation (HGU)	Core revenue center. Fractionates crude into LPG, BS-VI fuels, and naphtha. Daily crude assays, cut point margins, and catalyst deactivation rates are confidential trade secrets.
Petrochemicals Complex	Polypropylene Unit (PP - 440 KTPA), Propylene Recovery (PRU)	Upgrades cracked gases into polymer pellets. Highly exothermic kinetics. Production slates and customer pricing contracts are commercially sensitive.
Aromatics Complex	Naphtha Splitter, Parex, Isomar, High-Purity Benzene & Para-Xylene	Deep chemical transformation. Strict environmental and toxic aromatics monitoring. Catalyst yields and furnace run-lengths are proprietary.
Offsites & Utilities (O&U)	Captive Power Plant (82 MW), Steam Boilers (45/60 bar), Desalination, Flare	Refinery operational backbone. Utility failure trips the entire complex within 120 seconds. Steam balances, desalination, and flare emissions require 24x7 monitoring.
Statutory Safety & Audits	Ultrasonic Thickness (UT), API 510/570, ASME B31.3, OISD-118/156	Mandatory safety governance. Tracks corrosion degradation and relief valve sizing. Scanned inspection logs and thickness scans are strictly confidential and audited.

4. Technical Architecture & Multi-Agent Orchestration Core

DRISHTI-MRPL employs a sovereign microkernel multi-agent architecture designed to operate seamlessly on air-gapped hardware. The system decouples orchestration, data processing, document assembly, and sandboxed computing into specialized local workers:

Specialized Agent	Core Engine & Technologies	Air-Gapped Responsibility & Tool Capabilities
SupervisorAgent	ReAct Framework + Intent Router Python 3.12 Orchestrator	Autonomous task decomposition. Analyzes multi-modal prompts, selects execution steps, dynamically dispatches sub-tasks to specialized agents, and synthesizes final outputs.
DataAgent	DuckDB (In-Process SQL OLAP) Zero-External Network Engine	High-speed analytical processing of crude assays, production slates, and equipment inventory. Executes vectorized SQL queries over gigabytes of industrial tabular data in milliseconds.
DocumentAgent	BM25 Ranker + SQLite FTS5 Zero Cloud Embeddings	Deterministic retrieval across internal SOPs, API/ASME standards, and OISD safety manuals. Provides grounded citations without transmitting text to third-party embedding APIs.
SandboxAgent	AST-Restricted Python Runner In-Process Subprocess Isolation	Executes mathematical and engineering calculations (ASME B31.3 hoop stress, Barlow's formula, remaining life) in a locked execution sandbox blocking OS and network system calls.
TemplateAgent & Modifier	python-docx + python-pptx + openpyxl Institutional XML Synthesizers	Directly compiles and modifies formal PSU board approval notes (.docx), executive slide presentations (.pptx), and multi-sheet calculation workbooks (.xlsx) with embedded charts.
VisionAgent	Tesseract OCR + Local Qwen2.5-VL PyMuPDF (fitz) Image Rasterizer	Dual-layer raster analysis: extracts machine text and numbers via high-precision OCR, combined with VLM visual reasoning for P&ID flow arrows, schematics, and handwritten notes.
SovereignMonitor	psutil Network Sniffer + SHA-256 Cryptographic Audit Signer	Continuous hardware and network socket auditor. Monitors every process socket, confirms zero external WAN egress, and issues digitally signed audit certificates on request.

Dynamic Open-Weight Model Routing Matrix

To maximize inference speed and precision on workstation hardware (e.g., NVIDIA RTX 3060/4090 with 12GB to 24GB VRAM), incoming tasks are classified and routed to domain-optimized open-weight models via models_config.json:

Task Classification	Primary Open-Weight Model	VRAM Footprint	Functional Specialization & Benchmark
Complex Reasoning & Math	DeepSeek-R1-Distill-Qwen-14B (Q4_K_M)	8.9 GB	Multi-step engineering calculations, remaining equipment life derivations, and regulatory compliance checks.
Code, SQL & Automation	Qwen2.5-Coder-7B-Instruct (Q4_K_M)	4.7 GB	SQL query generation for DuckDB crude assays and Python scripts for engineering verification in the sandbox.
Multimodal Vision & Schematics	Qwen2.5-VL-7B-Instruct (Q4_K_M)	5.2 GB	Interpretation of CAD P&IDs, piping schematics, scanned inspection tables, and handwritten shift logs.
PSU Note & Slide Drafting	Llama-3.1-8B-Instruct (Q4_K_M)	4.9 GB	High stylistic fidelity for formal Indian PSU board approval memos, executive slide decks, and administrative correspondence.

5. Forensic Gap Analysis Matrix: Problem Statement vs. Prototype

A rigorous forensic audit was conducted comparing the SIH 2026 Problem Statement requirements against the current DRISHTI-MRPL prototype. The matrix below delineates the operational status, identifies specific missing user-facing elements, and rates the criticality of each gap from an evaluator's perspective:

#	Problem Statement Requirement	Current Prototype Status	Identified Gap & Missing Evaluator Element	Severity
1	**Verifiable Air-Gap Claim** Show via logs or visible network monitor that no external calls are made.	Sovereign_monitor.py tracks sockets and generates a cryptographic SHA-256 certificate via /api/certificate.	**No prominent real-time network monitor widget in the header.** The UI displays CPU/RAM/GPU tickers, but lacks a visible WAN: 0.00 KB/s badge and click-to-view live socket inspector.	HIGH
2	**Dynamic Model Auto-Routing** Automatically pick right model based on task; add new models without redesign.	Backend routes intents across Coder, Reasoning, Vision, and Drafting models via models_config.json.	**1. No routing breadcrumbs in chat.** User sees generic 'DRISHTI AI' without model attribution. **2. Settings tab has display-only table** without an interactive '+ Register Model' modal.	HIGH
3	**Visible Agentic Planning & Iteration** Plan multi-step work, invoke local tools, and iterate autonomously.	Supervisor agent decomposes tasks and executes multi-agent tool loops (DuckDB, Python sandbox, DOCX builder).	**ReAct execution trace is hidden in backend terminal logs.** The chat UI renders only the final synthesis, obscuring the multi-step autonomous tool pipeline from evaluators.	HIGH
4	**Tangible Deliverables (PPT/Word/Excel)** Real approval notes, PPT/Word/Excel files, calculations with steps shown.	Template_agent.py generates .docx memos and .pptx slide decks. Tested and verified in backend test suite.	**Missing 1-click PPTX generation chip and rich deliverable cards.** Chat bubbles lack visual download cards with file size, icon, and direct open buttons for generated slides.	HIGH
5	**Multimodal Ingestion (Handwritten/CAD)** Read scanned PDFs, handwritten notes, engineering drawings on-device.	Vision agent combines Tesseract OCR and Qwen-VL for CAD P&IDs and scanned ultrasonic inspection PDFs.	**Missing authentic handwritten shift log image.** Dataset includes clean CAD drawings and PDFs, but lacks a realistic handwritten maintenance log sample to prove cursive OCR.	HIGH
6	**Sandboxed Code Verification** Run and verify code in a secure sandbox with visible calculation steps.	Sandbox_agent.py executes mathematical derivations in an AST-restricted Python environment.	**Sandbox execution results are embedded in raw markdown text.** Lacks an explicit 'Sandboxed Execution Verification' badge proving mathematical safety and isolation.	MEDIUM
7	**Grounded Local Knowledge Base** Connect to internal manuals, SOPs, and past correspondence locally.	DocumentAgent indexes API 510, ASME B31.3, PFCCU operating manuals via BM25 and SQLite FTS5.	Fully functional. Minor enhancement: display interactive citation source chips (e.g., [API 510 Section 5.4]) that pop open the exact local excerpt.	LOW

EVALUATOR SCORING DYNAMICS & THE 'VISIBILITY' FACTOR

SIH evaluators judge working software in short evaluation intervals (3 to 5 minutes). Even if backend code is technically flawless, features that are not visibly represented in the UI receive zero credit. The backend currently implements 90% of the PS requirements, but the absence of visible air-gap monitors, model attribution badges, and ReAct thought traces makes it appear to casual observers like a standard chat interface. Converting backend telemetry into visible UI artifacts is paramount.

Deep Dive: Detailed Analysis of the Six Critical Gaps

To ensure complete compliance with every sentence of the SIH 2026 Problem Statement, the following six operational gaps have been analyzed in detail with specific technical remediation designs:

1. Real-Time Visible Air-Gap Network Monitor & Socket Auditor

• **Problem Statement Clause:** *'The system should also show, through logs or a visible network monitor, that no external calls are made at any point. That's the actual proof of the sovereign claim, not just a statement of it.'*
• **Current Reality:** Sovereign_monitor.py polls psutil net_connections and generates signed certificates via /api/certificate, but the UI only displays CPU/RAM/GPU counters in the header.
• **Remediation Design:** Add a prominent network telemetry pill in the header: WAN: 0.00 KB/s | LAN: 42 KB/s with a steady emerald lock. Clicking the pill opens a modal listing all open process sockets bound to 127.0.0.1 and a 1-click button to view the SHA-256 certificate.

2. Model Auto-Routing Breadcrumbs & Intent Attribution

• **Problem Statement Clause:** *'support multiple open weight models at once and automatically pick the right one for a given task based on what that task needs, a coding request handled differently from a document summary request.'*
• **Current Reality:** Backend routes intents across Coder, Reasoning, and Vision models, but the UI labels all answers uniformly as 'DRISHTI AI'.
• **Remediation Design:** Return model_used, intent_category, and confidence in the API response. Render a styled badge above every message bubble: [Auto-Routed: DeepSeek-R1-Distill-14B | Domain: Engineering Math | 98.4% Confidence].

3. Transparent ReAct Agent Execution Trace Accordion

• **Problem Statement Clause:** *'The assistant also needs to actually act like an agent. Plan out multi step work, call local tools... and iterate on a task instead of answering once and stopping.'*
• **Current Reality:** Supervisor_agent.py performs multi-step planning and tool invocation, but sends only the final synthesized markdown to the user.
• **Remediation Design:** Stream execution steps to the frontend and display a collapsible accordion: *'Agent Execution Trace (4 Steps Completed)'* showing: Step 1: Ingest Document -> Step 2: Query DuckDB -> Step 3: Run AST Sandbox -> Step 4: Assemble Deliverable.

4. 1-Click Executive Presentation (.pptx) & Styled Deliverable Cards

• **Problem Statement Clause:** *'Output should be real deliverables, approval notes, PPT/Word/Excel files, working code, calculations with steps shown, not just chat replies.'*
• **Current Reality:** Template_agent.py supports PPTX creation, but there is no prominent suggestion chip to demonstrate it, and deliverables appear as simple text links.
• **Remediation Design:** Add a prompt chip: *'Generate Executive Briefing Deck for CDU Revamp (.pptx)'*. Render styled deliverable cards in chat with file icons, byte sizes, and direct download links.

5. Authentic Handwritten Maintenance Log Ingestion Sample

• **Problem Statement Clause:** *'scanned PDFs, handwritten notes, engineering drawings, photographs, read through on device OCR and vision models.'*
• **Current Reality:** The dataset has CAD P&IDs and scanned inspection reports, but lacks an authentic handwritten shift log to showcase cursive handwriting recognition.
• **Remediation Design:** Synthesize an authentic handwritten refinery shift handover sheet (data/handwritten_shift_handover_log.png) detailing pump gland seal leaks and temperature spikes, supported by a pre-configured query.

6. Dynamic Open-Weight Model Registration Modal in Settings Tab

• **Problem Statement Clause:** *'New open weight models should be addable later without redesigning the system, since this space is moving fast.'*
• **Current Reality:** The Model Registry in tab-settings is a static HTML table. Administrators cannot register new models through the UI.
• **Remediation Design:** Add a **'+ Register New Open-Weight Model'** button and modal in Settings. Submitting the form writes directly to models_config.json, enabling dynamic model addition without server restarts.

6. Industrial Edge Cases, Safety & Fault Tolerances

Deploying generative agentic AI inside hazardous process industries (petroleum refineries, petrochemical plants, nuclear reactors) introduces severe failure modes that do not exist in conventional enterprise chat software. The DRISHTI-MRPL architecture implements strict defensive guardrails across five critical operational failure vectors:

Edge Case Dimension	Industrial Failure Scenario	Architectural Defense & Mitigation Mechanism
Multimodal Degraded Scans	• Carbon-copy shift notes with low optical contrast. • P&ID schematics with skewed lines and oil smudges. • Blurry ultrasonic thickness gauge readouts.	**Dual-Pass Hybrid OCR + VLM Verification:** Tesseract extracts high-confidence numeric characters, while Qwen-VL interprets topological connectivity. If OCR confidence drops below 65%, the system flags the parameter as 'Tentative - Requires Field Verification' rather than hallucinating critical dimensions.
Mathematical Hallucination in Engineering Calculations	• LLMs generating approximate or hallucinated values for pipe wall thickness, corrosion allowance, or relief valve sizing. • Catastrophic risk if applied to high-pressure lines.	**Mandatory Sandboxed Code Re-Computation:** The LLM is strictly prohibited from generating raw calculation numbers directly. It must formulate a deterministic Python formula (Barlow's formula, ASME B31.3 Section 304.1.2) which is executed by SandboxAgent. The computed result is injected back into the memo.
Sandbox Security & Code Injection	• Prompts attempting to execute shell commands, import socket libraries, access sensitive OS files, or trigger infinite loops / fork bombs.	**Static AST Analysis & Kernel Isolation:** The Python sandbox uses Abstract Syntax Tree parsing to block all import, os, sys, socket, eval, and exec calls. Execution is restricted to mathematical primitives with a strict 5.0-second CPU timeout and 128MB RAM quota.
Hardware & VRAM Saturation	• Heavy multi-agent query triggering concurrent LLM loads, exceeding 16GB VRAM on mid-range competition workstations and causing GPU OOM crashes.	**Sequential Agent Execution & Quantized Fallbacks:** Supervisor executes subagents in sequential pipeline phases rather than concurrent VRAM allocation. Models are quantized to 4-bit (GGUF Q4_K_M). If VRAM exceeds 92%, the system seamlessly spills to high-speed system RAM.
Air-Gap Integrity & DNS Leaks	• Third-party Python libraries (e.g. requests, urllib, pip) silently attempting background analytics, telemetry, or external DNS resolution.	**Strict Socket Whitelisting & Sovereign Sniffer:** All internal endpoints bind explicitly to 127.0.0.1. The frontend uses zero CDN links (all CSS, fonts, icons are local SVG). SovereignMonitor continuously verifies that socket destination IPs match loopback addresses.

INDUSTRIAL SAFETY INTERLOCKS & DETERMINISTIC FAIL-SAFES

In industrial process control, a wrong answer is infinitely worse than no answer. If an ultrasonic thickness reading indicates that a high-pressure line has worn beyond the statutory minimum retirement thickness (t_{min}), DRISHTI-MRPL enforces a deterministic safety interlock: it overrides any operational optimization suggestion and automatically generates a statutory OISD Level-1 Shutdown Alert. AI suggestions never bypass deterministic safety logic.

7. Concrete Engineering Implementation Roadmap

To elevate DRISHTI-MRPL from 90% prototype readiness to a flawless 100% compliant SIH 2026 benchmark, the following six engineering work packages have been structured for systematic execution:

Task 1: Real-Time Air-Gap Telemetry Widget & Socket Audit Modal

Target Files: static/index.html, static/app.js, static/app.css

• Add a live network telemetry pill in the header next to CPU/RAM: WAN: 0.00 KB/s | LAN: Active with an animated green lock.

• Implement a click handler that opens a *'Sovereign Air-Gap Security Audit'* modal displaying real-time open sockets, zero external IP connections, and a 1-click button to download the SHA-256 cryptographic certificate.

Task 2: Dynamic Model Routing Breadcrumbs & Attribution Badges

Target Files: static/app.js, static/app.css, Supervisor_agent.py

• Modify backend chat response JSON to return selected_model, task_domain, and routing_confidence.

• Render a styled model badge above every AI response in the chat window (e.g., [Routed to: DeepSeek-R1-Distill-14B | Domain: Engineering Math | Latency: 1.4s]).

Task 3: Visible ReAct Multi-Step Agent Execution Accordion

Target Files: Supervisor_agent.py, static/app.js, static/app.css

• Expose the internal supervisor execution plan in the streaming response payload.

• Render an interactive, collapsible accordion above the final answer in chat: *'Agent Execution Trace (4 Steps Completed)'* showing: Step 1: Ingest Document -> Step 2: Query DuckDB -> Step 3: Run AST Sandbox -> Step 4: Assemble Deliverable.

Task 4: 1-Click Executive Presentation (.pptx) Generator & Deliverable Cards

Target Files: static/index.html, static/app.js, Template_agent.py

• Add a prominent prompt suggestion chip: *'Generate Executive Briefing Deck for CDU Revamp (.pptx)'*.

• Update chat rendering to display rich deliverable cards for generated .docx, .pptx, and .xlsx files featuring file icons, byte sizes, and direct download links.

Task 5: Ingest Authentic Handwritten Maintenance Shift Handover Log

Target Files: data/handwritten_shift_handover_log.png, generate_real_datasets.py

• Synthesize and place a high-fidelity image of an authentic handwritten refinery shift handover log in data/ detailing pump seal leaks and temperature excursions.

• Add a quick prompt: *'Read handwritten shift log and verify if reported pump leaks require inventory spares'*.

Task 6: Interactive Open-Weight Model Registration Modal in Settings Tab

Target Files: static/index.html, static/app.js, server.py, models_config.json

• Add a **'+ Register New Open-Weight Model'** button to the Model Registry card in tab-settings.

• Build a clean modal allowing administrators to input model name, Ollama tag, domain specialization (Code, Math, Vision, Drafting), and VRAM profile, dynamically updating models_config.json without restarting the server.

Task Package	Codebase Files Impacted	Target Completion	SIH Evaluator Impact
P1: Air-Gap HUD & Modal	index.html, app.js, Sovereign_monitor.py	Immediate	**Critical (Air-Gap Proof)**
P2: ReAct Trace & Badges	app.js, app.css, Supervisor_agent.py	Immediate	**Critical (Agentic Proof)**
P3: PPTX Chips & Cards	index.html, app.js, Template_agent.py	Immediate	**High (Deliverables Proof)**
P4: Handwritten Sample	data/, generate_real_datasets.py	Immediate	**High (Multimodal Proof)**
P5: Dynamic Registry	index.html, app.js, server.py	Immediate	**High (Extensibility Proof)**

8. The Winning SIH 2026 Three-Act Demonstration Flow

During live hackathon evaluations, judges review projects in strict 3-to-5 minute evaluation windows. The demonstration script must be choreographed to deliver maximum technical impact and eliminate skepticism within the first 60 seconds:

Act & Duration	Evaluator Demonstration Workflow	Evaluator Impact & Proof Point
ACT 1: The Sovereignty & Air-Gap Proof (45 Seconds)	1. Disconnect physical Ethernet or toggle off Wi-Fi on the host machine. 2. Direct evaluator attention to the header telemetry widget showing WAN: 0.00 KB/s. 3. Click the widget to open the *Live Socket Audit* modal, proving all 7 active sockets are bound to 127.0.0.1. 4. Generate and display the SHA-256 Air-Gap Security Certificate.	Immediately establishes credibility. Eliminates skepticism about hidden cloud APIs (OpenAI/Anthropic) before the demo begins.
ACT 2: The Autonomous Multimodal Pipeline (90 Seconds)	1. Click the prompt: *'Review CDU ultrasonic thickness scan, calculate corrosion rate per API 510, and draft Board Approval Note'*. 2. Watch the visible ReAct trace expand: VisionAgent reads scanned PDF -> DataAgent queries baseline pipe schedules in DuckDB -> SandboxAgent executes Barlow's formula in Python -> TemplateAgent generates .docx memo. 3. Click the deliverable card to open the fully formatted Board Note in Microsoft Word.	Demonstrates the full agentic loop: multimodal vision + SQL OLAP + sandboxed code execution + professional corporate deliverable in under 15 seconds.
ACT 3: Dynamic Multi-Model Registration (45 Seconds)	1. Point out the model routing badge in the chat answer (auto-routed to DeepSeek-R1 for math). 2. Navigate to the **Settings Tab** and click *'+ Register New Open-Weight Model'*. 3. Register a hypothetical new model (e.g., mistral-large:24b for PSU policy synthesis). 4. Show that models_config.json is updated live without restarting the server.	Directly addresses the Problem Statement's requirement that the system must not be locked to one model and must accommodate rapidly evolving open-weight LLMs.

Evaluator Q&A Defense & Competitive Moats

Likely Evaluator Question	Authoritative Engineering Defense
'How do you prove you aren't calling OpenAI or Claude in the background?'	We run with the network interface unplugged. Our SovereignMonitor inspects active OS sockets via psutil every 2 seconds, proving 0 bytes WAN transmitted and generating a cryptographic SHA-256 audit certificate.
'What if the judge's venue doesn't have a $10,000 GPU server?'	The workbench runs entirely on consumer/workstation hardware (e.g. RTX 3060 12GB / RTX 4090). By using 4-bit quantized GGUF models and sequential agent activation, peak VRAM is kept under 9GB. If no GPU is present, it gracefully falls back to CPU inference.
'Why not just use a local Ollama WebUI wrapper?'	Ollama WebUI is a passive chat interface. DRISHTI-MRPL is an agentic workbench: it executes multi-step ReAct plans, queries DuckDB SQL databases, runs sandboxed Python code to prevent hallucinated engineering formulas, and directly outputs formatted Word and PowerPoint files.

EXECUTIVE CONCLUSION & DEPLOYMENT READINESS

DRISHTI-MRPL stands in the 95th percentile of readiness for the SIH 2026 Problem Statement. The underlying backend possesses every required agentic capability -- DuckDB analytics, AST sandboxed execution, deterministic document synthesis, multimodal vision, and socket-level air-gap auditing. By implementing the six visual and user-facing enhancements detailed in this report, the platform will deliver an unassailable, gold-standard demonstration for Smart India Hackathon evaluators.